

Client–Server Communication Protocol

AM48 Group — Brotto, Casagrande, Casati, Clerico
Software Engineering Final Project — A.Y. 2025/2026

Abstract

This document describes the communication protocol between client and server in the distributed implementation of *Mesos*. It explains how player actions are sent to the server, validated by the Model, and propagated to all participants as incremental state updates. The protocol is designed to be **transport-agnostic**: the same application logic travels identically over Socket and RMI. The main scenarios are illustrated through UML sequence diagrams.

Contents

1	Introduction	2
2	Message structure	3
3	Transport abstraction: Socket and RMI	4
4	The Delta model	5
5	Error handling	6
6	Scenarios	7
6.1	Connection and Join	7
6.2	Place Totem: standard action with multiple return	7
6.3	Exception: domain error	8
6.4	Take Card and automatic inline transitions	8

1 Introduction

The interaction between client and server is organised around two complementary message flows and one core principle: *the Model lives on the server* and is the single source of truth for the game state. Clients keep a local replica of the state, which is kept aligned through updates pushed by the server.

The core idea of the protocol is the following:

1. The client sends the server an **action request** (join, place totem, take card) through the `VirtualServer` interface.
2. The server receives the request and forwards it to the `GameController`, which identifies the game involved and updates its Model.
3. The Model validates the move according to the game rules. If the move is valid, it returns a list of incremental changes (`List<GameDelta>`); if it is illegal, it throws an `InvalidActionException`.
4. The server propagates the changes to all clients in the game (broadcast), or, in case of error, reports them to the sender only.
5. Each client applies the changes to its own local replica and refreshes the View.

Given the large number of possible actions and the fact that each is valid only in certain game phases, we chose an architecture in which validation resides entirely within the Model, while the network layer is limited to translating requests into Controller calls and distributing the responses. This keeps the network layer thin and free of domain logic.

2 Message structure

The protocol defines two families of messages, one per direction. On the Socket transport these are classes serialised/deserialised in JSON, while on RMI they correspond directly to remote method calls (see Section 3).

Commands (client $\rightarrow$ server). All messages directed to the server derive from the abstract class `ClientCommand`. Each concrete subclass represents a specific player action and is *self-contained*: once deserialised, it carries out its action through the `execute` method. The three game actions are `JoinGameCommand`, `PlaceTotemCommand` and `TakeCardCommand`.

Notifications (server $\rightarrow$ client). All messages directed to clients derive from the abstract class `ServerNotification`. Each subclass knows how to apply itself to the client's local replica (`ClientModel`) through the `apply` method. The three notifications are `GameDeltaNotification` (incremental update), `InitialSnapshotNotification` (full state) and `ErrorNotification` (error).

Polymorphic deserialization. Both hierarchies rely on Jackson's polymorphic deserialization: a "type" field in the JSON payload automatically selects the correct concrete subclass. This lets the receive loop deserialise any message with a single call, without manually inspecting its content.

Listing 1: Polymorphic deserialization configuration on `ClientCommand`.

```
@JsonTypeInfo(use = JsonTypeInfo.Id.NAME, property = "type")
@JsonSubTypes({
    @JsonSubTypes.Type(value = JoinGameCommand.class, name = "joinGame"),
    @JsonSubTypes.Type(value = PlaceTotemCommand.class, name = "placeTotem"),
    @JsonSubTypes.Type(value = TakeCardCommand.class, name = "takeCard")
})
public abstract class ClientCommand {
    public abstract void execute(SocketClientHandler handler);
}
```

The mirror hierarchy `ServerNotification` adopts the same mechanism, exposing the `apply(ClientModel model)` method that keeps the client-side receive loop simple: it deserialises the message and invokes `apply`, without any branching on the type.

Direction	Base class	Concrete subclasses
client $\rightarrow$ server	<code>ClientCommand</code>	<code>JoinGameCommand</code> , <code>PlaceTotemCommand</code> , <code>TakeCardCommand</code>
server $\rightarrow$ client	<code>ServerNotification</code>	<code>GameDeltaNotification</code> , <code>InitialSnapshotNotification</code> , <code>ErrorNotification</code>

Table 1: The two message hierarchies of the protocol.

3 Transport abstraction: Socket and RMI

The protocol supports two communication technologies, Socket and RMI, without duplicating the application logic. This is achieved by introducing two agnostic interfaces that form the boundary between game logic and transport mechanism:

- **VirtualServer**: the server as seen from the client. It exposes the three game actions (`joinGame`, `placeTotem`, `takeCard`) plus an optional `disconnect`. Implemented by `RmiClient` (RMI) and `SocketServerHandler` (Socket).
- **VirtualView**: a single client as seen from the server. It exposes the push methods toward the client (`showGameDelta`, `showInitialSnapshot`, `reportError`). Implemented by `RmiViewAdapter` (RMI) and by the server-side socket counterpart.

Thanks to these interfaces, the rest of the code (Controller, View, game management) does not know, and does not need to know, over which transport a message travels. The **logical flow shown in the sequence diagrams is identical for Socket and RMI**; only the underlying transport mechanism changes:

	Socket	RMI
Command send	JSON serialization of <code>ClientCommand</code>	remote method call
Notification push	JSON serialization of <code>ServerNotification</code>	remote callback invocation
Type validation	"type" field + Jackson	method signature (static)

Table 2: The two agnostic interfaces realised by transport-specific implementations.

4 The Delta model

A central design choice of the protocol is the propagation of updates through **incremental deltas** rather than re-sending the entire state on every move. After each valid action, the Model always returns a `List<GameDelta>`: each `GameDelta` describes only the fields that changed since the previous update, keeping the network payload small.

Why deltas and not the full snapshot. Sending the entire Model on every move would be simple but costly and redundant: most of the state does not change between one action and the next. The delta model reduces traffic and makes explicit *what* changed, simplifying the targeted update of the View. The full snapshot is used only where it is truly needed: at game start-up and during reconnection (Section 6.1).

Client-side application. On reception, each delta is applied to the local replica through `ClientModel.applyDelta`, which delegates to the delta itself (`delta.applyTo(this)`) and then notifies the observers to refresh the View. The method includes a guard for the edge case in which a delta arrives before the initial snapshot:

Listing 2: Client-side application of a delta, with a guard on the uninitialised state.

```
public void applyDelta(GameDelta delta) {
    if (state == null) {
        notifyError("Received game update before initial snapshot");
        return;
    }
    delta.applyTo(this);
    notifyObservers();
}
```

Automatic *inline* transitions. Since the Model always returns a *list*, automatic game transitions are expressed naturally by appending deltas to the same list. If an action completes a phase transition, the list includes the corresponding `PhaseChangedDelta`; if it triggers the end of the turn or of the game (EndTurn / EndGame), the relevant deltas are appended with no special network-side handling. At game end, `GameManager.takeCard` additionally appends a `LeaderboardDelta` with the historical results, which is also broadcast like the others (Section 6.4).

5 Error handling

Move validation is the exclusive responsibility of the Model. When a move violates a game rule (slot already occupied, wrong phase, insufficient resources, etc.), the Model throws an `InvalidActionException`. The exception propagates up to the server-side execution point, where it is caught and translated into an `ErrorNotification` sent **exclusively to the sender** through `reportError`. The other players receive nothing, since for them nothing has changed.

Listing 3: Error-handling pattern common to all commands (except from `PlaceTotemCommand`).

```
try {
    List<GameDelta> deltas =
        handler.getController().handlePlaceTotem(handler.getNickname(), position);
    List<String> recipients =
        handler.getController().getPlayersInGame(handler.getNickname());
    for (GameDelta d : deltas) {
        handler.getServer().broadcastToGame(recipients, d);
    }
} catch (Exception e) {
    handler.getSelfView().reportError(e.getMessage()); // sender only
}
```

On the client side, `reportError` translates into `ClientModel.notifyError`, which propagates the message to the registered observers through `onError`, causing the View to display it to the user.

6 Scenarios

The following sections illustrate the four main scenarios of the protocol through sequence diagrams. The `VirtualServer` and `VirtualView` lifelines represent the agnostic interfaces described in Section 3: the flow shown is identical for Socket and RMI.

6.1 Connection and Join

When a player requests to join a game, the command registers the client on the server and delegates to `handleJoinGame`, which returns a `JoinResult` object holding both the outcome (`gameStarted()`) and the snapshot to send. Two cases are distinguished:

- **Lobby not yet full** (`gameStarted == false`): the *partial* snapshot is sent with `showInitialSnapshot` to the *only* client that just connected, which displays the waiting screen.
- **Lobby full** (`gameStarted == true`): the Model runs `setupGame()`; the server retrieves the recipients with `getPlayersInGame` and sends the *full* snapshot to everyone through `broadcastSnapshotToGame`. The full snapshot overwrites any partial snapshot already present on the client side.

In both cases, on the client side `showInitialSnapshot` translates into `ClientModel.setInitialState`, which entirely replaces the local state and notifies the View with `onStateUpdated`.

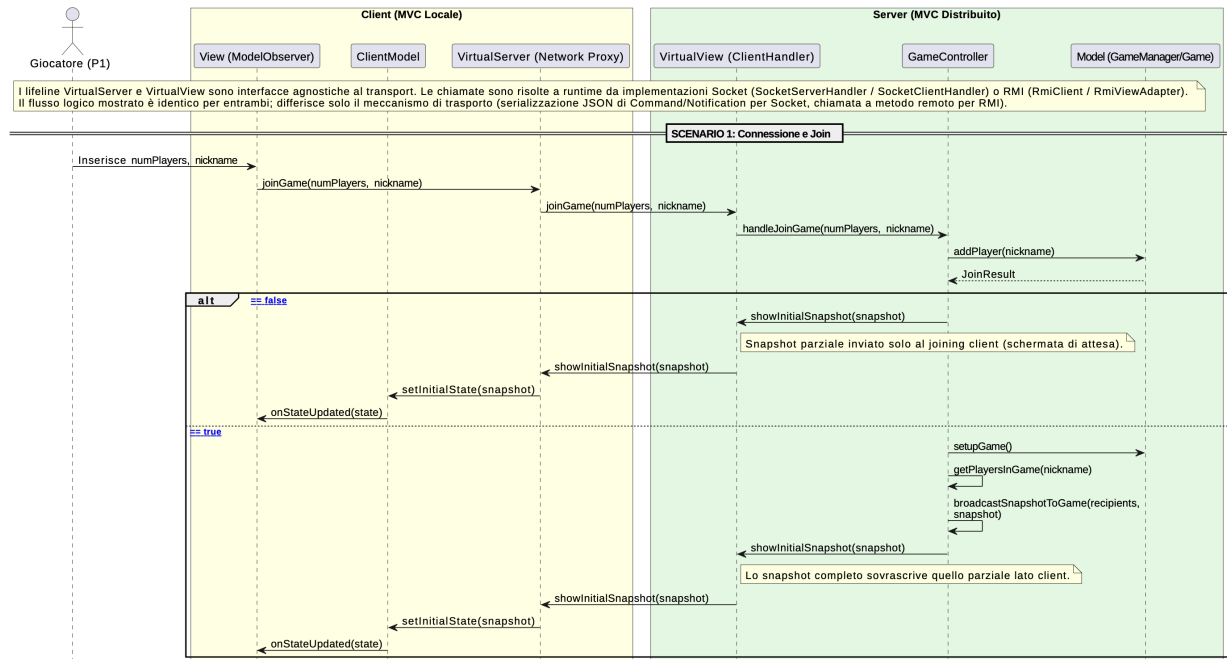


Figure 1: Scenario 1 — Connection and join. The `alt` block distinguishes the case of a lobby still waiting (partial snapshot to the joiner only) from that of a started game (full snapshot in broadcast).

6.2 Place Totem: standard action with multiple return

The totem-placement scenario illustrates the standard case of a valid action that produces a list of deltas. The command passes through `VirtualServer` → `GameController` → `Model`; the latter returns a `List<GameDelta>`. The server iterates over the list and broadcasts each delta to all players in the game with `broadcastToGame`; each client applies it with `applyDelta` and refreshes the View.

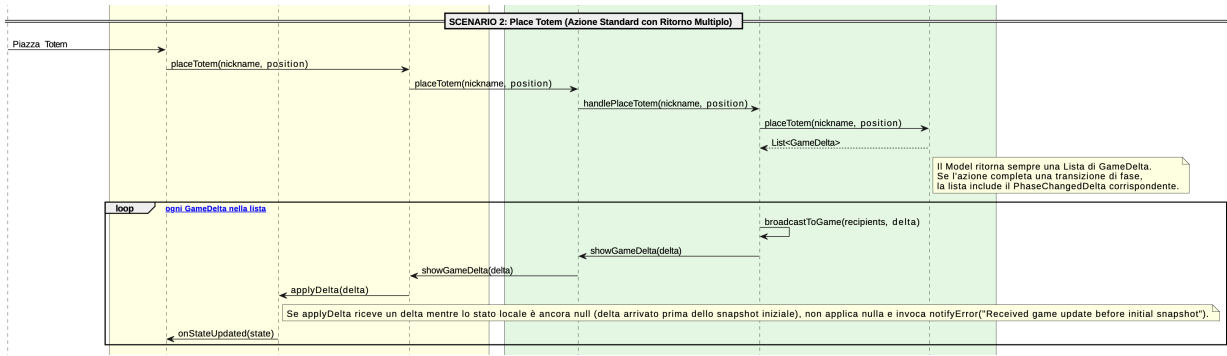


Figure 2: Scenario 2 — Place totem. The Model always returns a list of deltas; the loop block shows the broadcast iterated toward all clients.

6.3 Exception: domain error

This scenario shows the path of an illegal move. The initial flow is identical to Scenario 2, but the Model, having detected the rule violation, throws `InvalidActionException` instead of returning a list of deltas. The exception is caught on the server side and forwarded as `reportError` to the sender only; no broadcast is performed. On the client side the chain is `reportError` → `notifyError` → `onError`.

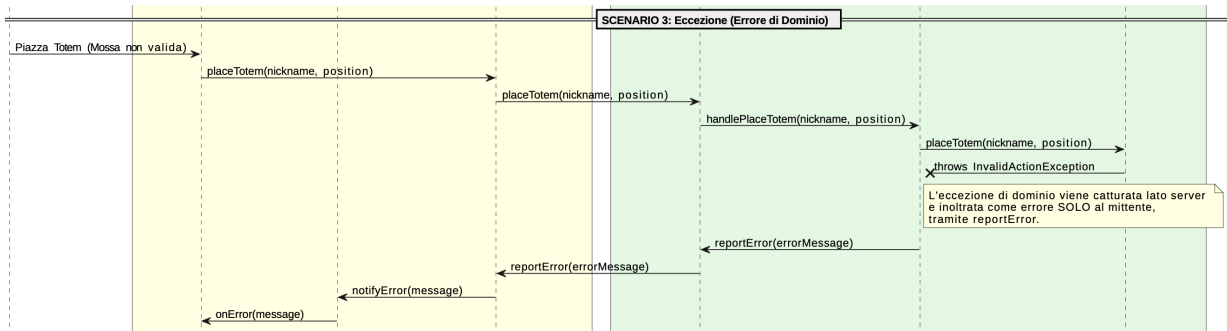


Figure 3: Scenario 3 — Domain error. The exception thrown by the Model is caught on the server side and reported exclusively to the player who attempted the move.

6.4 Take Card and automatic inline transitions

Taking a card is the most complete scenario, because it can trigger automatic transitions. An `alt` block distinguishes the two outcomes:

- **Validation failed:** the Model throws `InvalidActionException`, handled exactly as in Scenario 3 (error to the sender only).
- **Validation succeeded, with inline executions:** the Model returns the `List<GameDelta>`. If the action triggers `EndTurn` or `EndGame`, the relevant deltas are already appended to the same list; at game end the `LeaderboardDelta` with the historical results is also included. All deltas are broadcast in the same loop, with no special network-side handling.

This is where the delta model (Section 4) pays off the most: a single player action can translate into a coherent sequence of updates (take card, end turn, end game, leaderboard) propagated uniformly.

